Part 1 – Scenario 14: The Genome Archive

Scenario Explanation

The Genome Archive stores DNA sequences containing only four bases: A, T, C and G. The task is to generate a DNA sequence of 100,000 bases with a specified composition, construct a Huffman coding tree from the measured frequencies, encode the sequence, decode it by walking the tree, and verify that the original sequence is recovered. The experiment is also repeated for a uniform distribution of the four bases to determine when variable-length coding provides a useful compression benefit.

Chosen Technique and Justification

Chosen technique: Huffman coding

Huffman coding is appropriate because the DNA alphabet is small and the frequencies of the bases can be unequal. It assigns shorter codes to more frequent bases and longer codes to less frequent bases. For the AT-rich sequence, A and T occur much more frequently than C and G, so a variable-length code can reduce the average number of bits required per base. The generated prefix-free codes also allow unambiguous decoding by traversing the Huffman tree.

Rejected alternative: fixed-length coding

A fixed-length representation needs 2 bits per DNA base because there are four possible symbols. It is simple and has constant code length, but it cannot exploit unequal frequencies. Therefore, it is a useful baseline rather than the preferred compression method for an AT-rich sequence.

Algorithm/Logic

- Generate 100,000 DNA bases using the required probability distribution.
- Count the measured frequency of A, T, C and G.
- Create one Huffman leaf node for each DNA base and its measured frequency.
- Repeatedly combine the two lowest-frequency nodes to form the Huffman tree.
- Traverse the tree to generate a binary code for each base.
- Encode every DNA base using its Huffman code and calculate the total number of bits.
- Decode the bit string by walking from the root of the Huffman tree and verify the recovered sequence.
- Repeat the complete experiment with a uniform 25%, 25%, 25%, 25% distribution.

Complexity Analysis

Operation	Complexity	Explanation
Frequency counting	O(n)	Each of the n DNA bases is examined once to count the occurrences of A, T, C and G.
Huffman tree construction	$O(k^2)$ in this implementation	Repeatedly selects the two lowest-frequency nodes by scanning the node list and merges them until one root remains. Here, k = 4 because the DNA alphabet contains four bases.
Code generation	O(k)	The Huffman tree contains only four symbols, and the tree is traversed to generate one binary code for each base.
Encoding	O(n)	Each DNA base is processed once and replaced by its Huffman code.

Operation	Complexity	Explanation
Decoding	O(b)	Each encoded bit is processed once by walking through the Huffman tree, where b is the total number of encoded bits.
Space	O(n + k)	The program stores the DNA sequence and encoded bit string of size n, along with the Huffman tree containing k symbols.

Observation

The experimental results support the choice of Huffman coding for non-uniform DNA data. In the AT-rich experiment, A and T together account for approximately 80% of the sequence, so the Huffman tree assigns shorter codes to the frequent bases. The measured average is 1.798510 bits per base compared with the 2.00-bit fixed-length baseline, giving approximately 10.07% savings. In the uniform experiment, all four bases have nearly equal frequencies, and the Huffman tree assigns two bits to every symbol. Consequently, the average is exactly 2.000000 bits per base and there is no compression benefit. This shows that variable-length Huffman coding is worthwhile when the symbol distribution is sufficiently uneven, while fixed-length coding is equally efficient for a uniform four-symbol distribution.

Conclusion

Huffman coding successfully compressed and reconstructed both 100,000-base DNA sequences. The round-trip verification succeeded in both experiments. The AT-rich sequence demonstrated a measurable 10.07% reduction in average bits per base, whereas the uniform sequence showed no saving over the 2-bit fixed-length representation. Therefore, Huffman coding is most useful for DNA data with noticeably unequal base frequencies.